MENTIFY

Performance Test Results and Analysis

Development Branch | Local Integrated Environment

Execution date: 25 October 2026

Prepared for: SENG 34213 System Development Project

Executive Summary

This document records a performance and availability baseline for the Mentify application running locally from the development implementation. Testing covered the API gateway, the running backend service endpoints, the active frontend web server, and selected unauthenticated API paths. The measurements were executed against localhost so they represent the current developer-machine baseline, not production or staging capacity.

The gateway health endpoint sustained 1,000 requests at 25 concurrent clients with zero transport failures, 3,041.41 requests/second, mean request time 8.22 ms, 95th-percentile latency 42 ms, and maximum latency 88 ms. The frontend home page sustained 1,000 requests at 25 concurrent clients with zero transport failures, 97.67 requests/second, mean request time 255.95 ms, 95th-percentile latency 378 ms, and maximum latency 558 ms.

All checked service health endpoints returned HTTP 200/UP except the enrollment service on port 8083, which returned HTTP 401 for the health request. The unauthenticated login probe returned HTTP 500. These are functional or configuration findings and must be resolved or explicitly accepted before claiming end-to-end readiness.

1. Purpose and Guide Alignment

The SENG 34213 course guideline identifies “Performance Test Results” as a required testing artefact and requires the final development report to include performance results. This document is structured to provide the required evidence: test scope, environment, execution method, test cases, actual output, status, analysis, limitations, and recommendations.

This report is an execution record. It does not replace the guide’s separate requirements for test coverage, security evidence, CI/CD evidence, staging demonstration, or the final development report.

2. System Under Test

Item	Detail
Backend repository	https://github.com/SENG-34213/Mentify-backend
Backend branch checked	development
Backend commit checked	4913c96

Item	Detail
Backend workspace state	Clean; local branch tracks origin/development
Frontend repository	https://github.com/SENG-34213/frontend
Frontend branch (local checkout)	main
Frontend commit checked	8d73c69

Note: the requested development branch was verified for the backend. The available frontend checkout was on main at the time of execution; frontend results are therefore labelled as the currently running local frontend baseline rather than a verified frontend development-branch result.

Architecture observed from the project repository

- Spring Cloud API gateway on port 8080.
- Multiple Spring Boot domain services using service discovery/configuration.
- Next.js frontend application served locally on port 3000.
- Supporting local infrastructure included MongoDB, RabbitMQ, Redis/AI tooling, and other application dependencies visible as active processes.

3. Test Environment

Parameter	Value
Execution host	Mac16,1
CPU	10 logical cores
Memory	16.0 GB
Client location	Same host as the application
Network path	Loopback/localhost
Backend public entry point	http://127.0.0.1:8080
Frontend entry point	http://127.0.0.1:3000
Load generator	ApacheBench (ab)
Date/time basis	Local client date, 25 October 2026

The application was not restarted for this test. Tests were run against the already-running local processes supplied by the user. Database contents, cache state, JVM warm-up state, background consumers, and other local workload were not reset between individual probes.

4. Test Method

The test method used HTTP requests against stable, read-only or health-oriented paths so that performance testing would not create or modify user, course, enrollment, payment, assignment, attendance, or communication data.

Load profiles

- Smoke/availability probes: one request per endpoint using curl with a five-second timeout.
- API gateway load test: 1,000 total requests, 25 concurrent clients.
- Frontend load test: 1,000 total requests, 25 concurrent clients.

- Selected API path checks: 500 total requests per path, 10 concurrent clients.
- Metrics recorded: HTTP status, completed requests, failed transport requests, non-2xx responses, requests per second, mean latency, p95 latency, p99 latency, maximum latency, and response size where applicable.

Interpretation

- **"Failed requests"** means the load generator could not complete the HTTP exchange.
- **"Non-2xx responses"** means the server completed the exchange but returned a non-success status. Authentication failures are not treated as transport failures.
- The measurements are baselines, not service-level objectives, because the guide does not provide numeric latency or throughput targets.

5. Availability and Health Results

Test ID	Target	Expected	Actual	Status
PT-AV-01	GET http://127.0.0.1:8080/actuator/health	HTTP 200 and UP	HTTP 200, {"status":"UP"}	PASS
PT-AV-02	GET http://127.0.0.1:8081/actuator/health	HTTP 200 and UP	HTTP 200, {"status":"UP"}	PASS
PT-AV-03	GET http://127.0.0.1:8082/actuator/health	HTTP 200 and UP	HTTP 200, {"status":"UP"}	PASS
PT-AV-04	GET http://127.0.0.1:8083/actuator/health	HTTP 200 and UP	HTTP 401	FAIL / CONFIGURATION FINDING
PT-AV-05	GET http://127.0.0.1:8084/actuator/health	HTTP 200 and UP	HTTP 200, {"status":"UP"}	PASS
PT-AV-06	GET http://127.0.0.1:8085/actuator/health	HTTP 200 and UP	HTTP 200, {"status":"UP"}	PASS
PT-AV-07	GET http://127.0.0.1:8086/actuator/health	HTTP 200 and UP	HTTP 200, {"status":"UP"}	PASS
PT-AV-08	GET http://127.0.0.1:8087/actuator/health	HTTP 200 and UP	HTTP 200, {"status":"UP"}	PASS
PT-AV-09	GET http://127.0.0.1:8088/actuator/health	HTTP 200 and UP	HTTP 200, {"status":"UP"}	PASS
PT-AV-10	GET http://127.0.0.1:8089/actuator/health	HTTP 200 and UP	HTTP 200, {"status":"UP"}	PASS
PT-AV-11	GET http://127.0.0.1:3000/	HTTP 200 and HTML document	HTTP 200 and HTML document	PASS

Health summary: 10 of 11 checked application endpoints met the expected unauthenticated health/home response. The enrollment service health path on port 8083 requires investigation because its health endpoint is protected while the project setup guide describes health verification as an operational check.

6. Load Test Results

6.1 API Gateway health

Target: GET http://127.0.0.1:8080/actuator/health

Profile: 1,000 requests; concurrency 25

Metric	Result
Completed requests	1,000
Transport failures	0
Non-2xx responses	0
Requests per second	3,041.41
Mean request time	8.22 ms
P95 latency	42 ms
P99 latency	78 ms
Maximum latency	88 ms
Transfer rate	1,196.96 KB/s
Status	PASS for baseline availability/performance

The gateway health endpoint remained responsive under this short local burst. The tail latency was higher than the mean, but the maximum stayed below 100 ms during this run.

6.2 Frontend home page

Target: GET http://127.0.0.1:3000/

Profile: 1,000 requests; concurrency 25

Metric	Result
Completed requests	1,000
Transport failures	0
Non-2xx responses	0
Requests per second	97.67
Mean request time	255.95 ms
P95 latency	378 ms
P99 latency	547 ms
Maximum latency	558 ms
Transfer rate	1,448.04 KB/s
Status	PASS for baseline availability; OPTIMISATION RECOMMENDED

The frontend was available and completed the load without transport errors. Its mean and tail latency were materially higher than the gateway health endpoint, which is expected for server-rendered HTML and asset/reference generation, but warrants a separate browser-level test for user-perceived performance.

6.3 Selected API paths without authentication

The following checks intentionally omitted a bearer token to validate the security boundary and measure rejection speed. A non-2xx result is expected for protected endpoints.

Target	Profile	Completed	Transport failures	Non-2xx	Mean	P95	Status
GET /api/v1/auth/.well-known/jwks.json	500 / c10	500	0	500	0.866 ms	1 ms	FAIL: unexpected 401 for expected public key path

Target	Profile	Completed	Transport failures	Non-2xx	Mean	P95	Status
GET /api/v1/marketplace/materials	500 / c10	500	0	500	2.674 ms	10 ms	PASS: protected rejection is fast and consistent
GET /api/v1/course	500 / c10	500	0	500	1.585 ms	2 ms	PASS: protected rejection is fast and consistent

The protected endpoints returned quickly and consistently with HTTP 401. The JWKS path also returned HTTP 401, which is inconsistent with the expected public authentication-key discovery path used by resource-server validation and should be checked before staging.

6.4 Authentication smoke probe

Target: GET/POST probe of http://127.0.0.1:8080/api/v1/auth/login without valid credentials/payload

Observed result: HTTP 500 in 406.749 ms with response body

```
{"statusCode":500,"message":"Something went wrong. Please try again later."}
```

Status: **FAIL / FUNCTIONAL BASELINE FINDING**

This result is not a capacity failure by itself. It shows that the current local login path produces an internal server error for the probe used. A valid credential test with a controlled test account is required to determine whether the error is caused by malformed input, missing local configuration, an unavailable dependency, or an application defect. The report does not claim successful authenticated end-to-end performance because no valid demo credentials were provided.

7. Resource Observation

At the time of the final snapshot, the primary observed backend Java process used approximately 0.5% CPU, 0.3% memory, and 50,448 KB resident memory. This is a point-in-time observation, not a peak or average resource profile. The load tests were short and localhost-bound; they are insufficient for conclusions about production CPU, heap, database, queue, or network saturation.

8. Analysis

Strengths

- The API gateway health path completed 1,000/1,000 requests without transport failures or non-2xx responses.
- The frontend home page completed 1,000/1,000 requests without transport failures.
- Protected API paths rejected unauthenticated traffic consistently and quickly with HTTP 401.
- The running local environment had the expected gateway/frontend shape needed for repeatable baseline testing.

Findings requiring action

- Enrollment health on port 8083 returned 401 instead of an operational health response. Review service security configuration and health endpoint exposure.
- The public JWKS endpoint returned 401. Verify gateway routing and the intended unauthenticated allow-list; this can prevent resource services from validating tokens.
- The login probe returned 500. Execute a valid login test with a dedicated local test account and inspect auth-service/gateway logs.

- Frontend results are from the currently running local frontend checkout on main, not a verified development branch. Re-run after checking out the requested branch.
- No authenticated business-flow performance results were collected because valid credentials and safe test data were not supplied. The measured protected-path results must not be presented as business transaction performance.
- No browser-level Core Web Vitals, JavaScript execution, database query latency, JVM heap/GC, Redis, RabbitMQ, or external integration measurements were available from these short HTTP tests.

9. Recommended Next Test Cycle

1. Correct or document the enrollment health endpoint policy and make the intended readiness check return a deterministic status.
2. Verify that the JWKS endpoint is publicly reachable through the gateway and add an automated smoke test for it.
3. Run a valid login flow using a non-production demo account, then capture authenticated read-only flows for student, teacher, and admin roles.
4. Add authenticated scenarios for course listing, dashboard loading, announcements, notifications, assignments, attendance, and communication, using seeded test data and read-only operations.
5. Repeat each scenario at concurrency levels 1, 10, 25, 50, and 100 with a warm-up period and at least a five-minute sustained run.
6. Capture p50, p90, p95, p99, error rate, throughput, CPU, heap/GC, database connection-pool usage, database slow queries, Redis hit/miss ratio, and message-queue lag.
7. Execute the performance suite on the staging environment required by the course guide, because localhost results are not evidence of deployed capacity.
8. Store the command output and raw result files under documents/testing and link the final performance report from the development report appendix.

10. Reproducibility Commands

Gateway baseline

```
ab -n 1000 -c 25 -q http://127.0.0.1:8080/actuator/health
```

Frontend baseline

```
ab -n 1000 -c 25 -q http://127.0.0.1:3000/
```

Protected API rejection checks

```
ab -n 500 -c 10 -q http://127.0.0.1:8080/api/v1/marketplace/materials
ab -n 500 -c 10 -q http://127.0.0.1:8080/api/v1/course
```

Health smoke check

```
curl -i --max-time 5 http://127.0.0.1:<port>/actuator/health
```

11. Conclusion

The local Mentify environment was sufficiently live to execute a meaningful baseline. Gateway and frontend availability under short burst load were successful, with the gateway showing strong local response characteristics and the frontend showing stable but slower server-rendered response times. The application is not yet suitable for an unqualified “all systems pass” performance conclusion because the enrollment health endpoint returned 401, the JWKS path returned 401, and the login probe returned 500.

These findings should be resolved and the authenticated business workflows re-tested on the requested development branches and, finally, on staging.

Appendix A – Guide-Required Evidence Mapping

Guide requirement	Evidence in this document
Performance Test Results artefact	Sections 1–11; dedicated performance results document.
Testing strategy and test case documentation	Sections 4–6, including test IDs, expected output, actual output, and status.
Performance testing during testing/hardening	Executed load profiles and analysis in Sections 6–9.
Final report testing section includes performance results	This document can be referenced as the performance-results subsection/appendix of the final development report.
Honest limitations and known defects	Sections 6.4, 7, 8, and 11.

Appendix B – Test Execution Checklist

Result	Item
[PASS]	Local gateway reachable on port 8080
[PASS]	Local frontend reachable on port 3000
[PASS]	Gateway health load test completed
[PASS]	Frontend home load test completed
[PASS]	Protected endpoint rejection tests completed
[FAIL]	Enrollment health returned HTTP 401
[FAIL]	JWKS path returned HTTP 401
[FAIL]	Login smoke probe returned HTTP 500
[BLOCKED]	Authenticated business-flow performance tests require valid demo credentials and safe seeded data

Appendix C – Performance Improvement Branch Update

Item	Detail
Branch	perf/local-performance-improvements (created from development)
Starting commit	4913c96
Improvement commit	aceae80
Change implemented	Added /api/v1/auth/.well-known/jwks.json to the API gateway public security allow-list and added a regression test confirming anonymous JWKS access.

Verification

- Gateway compile: PASS.
- API gateway test suite: PASS; 12 tests, 0 failures, 0 errors, 0 skipped.
- Original running gateway on port 8080 before branch restart: JWKS returned HTTP 401, 0-byte response.
- Branch gateway started separately on port 18080: health returned HTTP 200 and JWKS returned HTTP 200 with a 423-byte JSON response.

- Branch JWKS load check: 500 requests at concurrency 10, 0 transport failures when variable response length was allowed, 954.19 requests/second, mean 10.48 ms, P95 32 ms, P99 54 ms, maximum 55 ms.
- Branch gateway health check: 1,000 requests at concurrency 25, 0 transport failures, 655.68 requests/second, mean 38.13 ms, P95 118 ms, P99 190 ms, maximum 219 ms.

Interpretation: the branch fixes the observed JWKS accessibility defect. The branch health throughput is lower than the earlier 3,041.41 requests/second baseline because it was measured in a separately started gateway process with different warm-up/logging/runtime conditions; it should not be interpreted as a regression until both versions are restarted and tested under identical conditions. The login HTTP 500 and authenticated business-flow testing remain open items requiring a valid local test account and payload.

Appendix D – Branch Performance Test Execution

Item	Detail
Execution date	25 October 2026
Backend branch	perf/local-performance-improvements
Branch HEAD at execution	59beeba
Performance fix included	aceae80
Branch gateway test port	18080
Existing application ports	Gateway 8080 and frontend 3000 were left running separately.
Load generator	ApacheBench

Warm branch gateway results after startup warm-up

Target	Profile	Failures	Req/s	Mean	P95	P99	Max	Notes
GET /actuator/health	1,000 / c25	0	1,588.68	15.736 ms	35 ms	76 ms	83 ms	—
GET /api/v1/auth/.well-known/jwks.json	500 / c10	0	761.44	13.133 ms	29 ms	62 ms	69 ms	ab run with -l
GET /api/v1/marketplace/materials (no token)	500 / c10	0	4,258.69	2.348 ms	4 ms	5 ms	36 ms	500 × HTTP 401
GET /api/v1/course (no token)	500 / c10	0	2,262.93	4.419 ms	11 ms	16 ms	16 ms	500 × HTTP 401
Frontend GET /	1,000 / c25	0	48.63	514.035 ms	990 ms	1,272 ms	1,405 ms	—

ApacheBench was run with variable response length enabled for the JWKS check because key JSON length can vary between responses.

Functional smoke result

- POST /api/v1/auth/login with an intentionally invalid test identity returned HTTP 400 “Invalid email or password” in 567.864 ms. This is a controlled validation response, not a valid-login performance result.

Branch conclusion

The branch improvement is verified: the public JWKS endpoint returned HTTP 200 and sustained 500/500 completed requests under the warm load test. Protected endpoints continued to reject anonymous requests consistently with HTTP 401. The frontend remains the main performance optimization area in this

local run because its P95 latency was 990 ms and P99 latency was 1,272 ms. Authenticated business-flow testing remains pending a valid temporary local account and seeded read-only data.

Reproducibility commands

```
ab -n 1000 -c 25 -q http://127.0.0.1:18080/actuator/health
ab -n 500 -c 10 -l -q http://127.0.0.1:18080/api/v1/auth/.well-known/jwks.json
ab -n 500 -c 10 -q http://127.0.0.1:18080/api/v1/marketplace/materials
ab -n 500 -c 10 -q http://127.0.0.1:18080/api/v1/course
ab -n 1000 -c 25 -q http://127.0.0.1:3000/
```